

Foundations: Decision Making

MDP theory

14 July 2026

Contents

1 Part 6 — Deep Foundation & Core Decision-Making Pillars	1
1.1 6.1 What a Markov Decision Process <i>is</i> (formal template)	1
1.2 6.2 The two Bellman equations, and the difference that actually matters	3
1.3 6.3 Why value iteration converges	4
1.4 6.4 Model-based vs model-free, stated sharply	6
1.5 6.5 Q-learning	7
1.6 6.6 Convergence conditions for Q-learning	8
1.7 6.7 Exploration vs exploitation in RL	9
1.8 6.8 The discount factor as an effective horizon	11
1.9 6.9 Certainty equivalence: the bridge, and the headline result	12
1.106.10 Side-by-side summary table	14
1.116.11 Where this connects to the swarm half	15

1 Part 6 — Deep Foundation & Core Decision-Making Pillars

Self-contained theory of sequential decision-making under uncertainty, written to be defended line-by-line in a viva and to seed the arXiv “Learning Journey” report. Notation is fixed once in §6.1 and reused throughout. Every update equation is attributed to its seminal source. This is the mirror of Part 1: where Part 1 searched a continuous space with a population, this searches a policy space with dynamic programming and with samples.

All numbers quoted are from `src/rl_water_tank.py` and the experiment driver; nothing here is illustrative.

1.1 6.1 What a Markov Decision Process *is* (formal template)

1.1.1 6.1.1 The five-tuple

A finite, discounted, infinite-horizon MDP is

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

with

- $\mathcal{S}$ — a finite set of **states**. Ours is $s = (L, t, g, F)$: tank level $L \in \{0, \dots, 10\}$ (one unit = 100 L), hour of day $t \in \{0, \dots, 23\}$, grid up/down $g \in \{0, 1\}$, diesel generator-hours remaining $F \in \{0, 1, 2\}$. So $|\mathcal{S}| = 11 \times 24 \times 2 \times 3 = 1584$.
- $\mathcal{A}$ — a finite set of **actions**, here $\{\text{IDLE}, \text{PUMP_GRID}, \text{PUMP_GEN}\}$, $|\mathcal{A}| = 3$. Availability is state-dependent: we write $\mathcal{A}(s) \subseteq \mathcal{A}$ and enforce it with a boolean mask (`_build_availability`). `PUMP_GRID` is not offered when $g = 0$ and `PUMP_GEN` is not offered when $F = 0$. That leaves **3432 legal** (s, a) **pairs out of** $1584 \times 3 = 4752$.

- P — the **transition kernel** $P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$, a conditional distribution over $\mathcal{S}$ for each legal (s, a) . We store it as a sparse $(|\mathcal{S}||\mathcal{A}|) \times |\mathcal{S}|$ CSR matrix, row index $s|\mathcal{A}| + a$; each row has at most $(D_{\max} + 1) \times 2 = 14$ non-zeros, so the dense array would be about 99.9% zeros.
- R — the **expected reward** $R(s, a) = \mathbb{E}[r_t \mid S_t = s, A_t = a] \in \mathbb{R}$. Ours is a cost, so $R \leq 0$ everywhere.
- $\gamma \in [0, 1)$ — the **discount factor**, here 0.99.

The **return** from time t is $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, and a (stationary, deterministic) **policy** is a map $\pi : \mathcal{S} \rightarrow \mathcal{A}$ with $\pi(s) \in \mathcal{A}(s)$. Its **value function** is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s \right], \quad Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s, A_t = a \right].$$

Because $|r| \leq r_{\max}$ (bounded below by $-(c_{\text{gen}} + c_{\text{ovf}} \cdot \text{pump} + c_{\text{short}} \cdot D_{\max}) = -127.5$ in our costs) and $\gamma < 1$, the series converges absolutely: $|V^\pi| \leq r_{\max}/(1 - \gamma)$. Boundedness is not decoration; it is what makes every operator below well-defined.

1.1.2 6.1.2 The disturbance form, and why our reward is not $r(s, a, s')$

The textbook writes $r = r(s, a, s')$. Ours cannot be written that way, and saying so is not a defect — it is the more general **disturbance form**

$$s_{t+1} = f(s_t, a_t, w_t), \quad r_t = r(s_t, a_t, w_t), \quad w_t \sim W(\cdot \mid s_t),$$

with disturbance $w_t = (D_t, g_{t+1})$: realised hourly demand and the next grid state. The realised shortage is $U_t = \max(0, D_t - \text{level after pumping})$, and once the tank hits zero the successor state s' no longer records *how much* demand went unmet — a demand of 4 and a demand of 6 against an empty tank both land in $L' = 0$. So D_t is not recoverable from s' and r is genuinely a function of the disturbance, not of the successor.

Nothing breaks. Value iteration only ever needs the **expected** reward

$$R(s, a) = \mathbb{E}_w[r(s, a, w)] = -\left(c_{\text{pump}}(a) + c_{\text{ovf}} \cdot \text{overflow} + c_{\text{short}} \cdot \sum_d p_t(d) \max(0, d - \ell_a)\right),$$

which is exactly what `build_model` tabulates (ℓ_a = level after pumping, p_t = the hour- t truncated-Poisson demand pmf). The Bellman operator is still a γ -contraction (§6.3), because the contraction argument never touches the *shape* of the reward, only its boundedness.

Q-learning, by contrast, consumes the **realised** r from `simulate_hour`. It must. If we fed it $\mathbb{E}_w[r]$ we would have leaked the demand distribution into the “model-free” agent and the whole comparison would be a fiction. `simulate_hour` never takes an expectation; `build_model` never takes a sample. Keeping those two faces honest is the single most load-bearing design decision in the code, and there is a Monte-Carlo parity test that estimates P and R back out of the simulator and asserts they match the tabulated model within sampling error.

1.1.3 6.1.3 The Markov property, stated precisely

The state is **Markov** if, for all t and all histories $h_t = (s_0, a_0, r_0, \dots, s_t)$,

$$\Pr(S_{t+1} = s', r_t = r \mid S_t = s_t, A_t = a_t, S_{t-1}, A_{t-1}, \dots, S_0) = \Pr(S_{t+1} = s', r_t = r \mid S_t = s_t, A_t = a_t).$$

In words: the current state screens off the entire past. Given (s, a) , the distribution of what happens next carries no extra information about how you arrived.

Our state satisfies this **by construction, and only because we built it to**:

- Demand D_t depends on the hour, so the hour t is *in* the state. Drop it and D becomes serially informative about itself through the diurnal cycle, and the process is no longer Markov in the remaining variables.
- Load-shedding is a two-state Markov chain $g_t \rightarrow g_{t+1}$ with hour-dependent $p_{\text{fail}}, p_{\text{restore}}$. Evening outages are both likely ($p_{\text{fail}} = 0.35$) and long ($p_{\text{restore}} = 0.15$, so $1/0.15 \approx 6.7$ h expected). The *duration* memory is encoded entirely in the binary g — which is exactly what a two-state chain buys you: “the power has been out for three hours” tells you nothing beyond “the power is out”.
- The diesel ration F must be in the state because burning fuel at 7am changes what is possible at 9pm. Without F the process has hidden memory and the whole “spend the ration wisely” story is unrepresentable.

This is the **only** thing that licenses both algorithms in this half of the lab. The Bellman equations are a statement about a one-step recursion; they are valid iff the one-step recursion is *complete*, which is precisely the Markov property. Value iteration inherits it directly (the backup conditions on (s, a) and nothing else). Q-learning inherits it too, and less obviously: the TD target $r + \gamma \max_{a'} Q(s', a')$ treats s' as a sufficient statistic for the future, so if s' is *not* Markov, the target is bootstrapping off a value that does not exist and the fixed point Q-learning converges to is not the value of anything. Non-Markov states break model-free learning quietly, which is worse than breaking it loudly.

Adding a variable to the state to restore Markovianity is always possible in principle (take the whole history as the state) and almost never possible in practice (the state space explodes). The engineering skill here is finding the *smallest* sufficient statistic. Ours is (L, t, g, F) at 1584 states.

1.1.4 6.1.4 Continuing, not episodic

The hour wraps: $t' = (t + 1) \bmod 24$. There is no terminal state. The 24-hour “episode” in `q_learning` is a **reporting unit**, not an episode in the MDP sense — the last update of each simulated day bootstraps normally off $\max_{a'} Q(s', a')$ and we never zero the value at the cut. Zeroing it would silently convert the problem into a finite-horizon one, and the Q that converged would not be the infinite-horizon Q^* that VI computes. The two curves in the report would then be comparing different objects and the comparison would mean nothing.

1.2 6.2 The two Bellman equations, and the difference that actually matters

Source. R. E. Bellman, *Dynamic Programming*, Princeton Univ. Press, 1957; R. A. Howard, *Dynamic Programming and Markov Processes*, MIT Press, 1960. The modern reference treatment is M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994, DOI [10.1002/9780470316887](https://doi.org/10.1002/9780470316887).

1.2.1 6.2.1 The expectation equation — linear, so solve it

Fix a policy π . Condition on the first step and use the Markov property:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) V^\pi(s').$$

Collect the $|\mathcal{S}|$ equations into vectors: let $R_\pi \in \mathbb{R}^{|\mathcal{S}|}$ with $(R_\pi)_s = R(s, \pi(s))$, and $P_\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ with $(P_\pi)_{s,s'} = P(s' \mid s, \pi(s))$. Then

$$V^\pi = R_\pi + \gamma P_\pi V^\pi \quad \Longleftrightarrow \quad (I - \gamma P_\pi) V^\pi = R_\pi.$$

This is a **linear system in V^π** . P_π is row-stochastic, so its spectral radius is exactly 1, so $\rho(\gamma P_\pi) = \gamma < 1$, so 1 is not an eigenvalue of γP_π , so $(I - \gamma P_\pi)$ is invertible and

$$V^\pi = (I - \gamma P_\pi)^{-1} R_\pi = \sum_{k=0}^{\infty} \gamma^k P_\pi^k R_\pi$$

(the Neumann series, which converges for the same reason). The point that gets missed: because the equation is linear, **you do not have to iterate it**. Iterating $V \leftarrow R_\pi + \gamma P_\pi V$ converges geometrically at rate γ and is what most student code does; solving the sparse system is exact and, at $|\mathcal{S}| = 1584$, faster.

Our policy_evaluation does exactly this:

```
P_pi = P[np.arange(S) * A + policy] # (S, S) sparse
R_pi = R[np.arange(S), policy]
return spla.spsolve(sp.eye(S, format="csr") - gamma * P_pi, R_pi)
```

One spsolve, no loop, no tolerance, no residual. This is the scoring function for **every** policy in the report — value iteration's, Q-learning's, certainty-equivalence's, the myopic control's, and the tuned caretaker heuristic's — and always under the **true** P . That removes Monte-Carlo noise from the comparison completely: policies are ranked by exact expected discounted return, not by which one drew a luckier rollout. Every regret number below is an exact quantity, not an estimate, apart from the seed-to-seed spread of the *learning* runs themselves.

1.2.2 6.2.2 The optimality equation — non-linear, so iterate it

Now do not fix a policy; ask for the best one.

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right], \quad Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a' \in \mathcal{A}(s')} Q^*(s, a')$$

The max is what changes everything. $V \mapsto \max_a [\dots]$ is not a linear map — it is piecewise-linear and convex — so there is no matrix to invert. There is no closed form. You cannot solve it; you can only find its fixed point, and the standard way is to apply the operator over and over. (There is an exact route: the optimality equation can be written as a linear program with $|\mathcal{S}|$ variables and $|\mathcal{S}||\mathcal{A}|$ constraints, minimise $\sum_s V(s)$ subject to $V(s) \geq R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s')$ for all (s, a) . It is exact and it is slower than VI at this size, so we do not use it. Mentioning it matters because “non-linear, therefore no exact method” would be false.)

The two equations differ in exactly one symbol and it is the whole difference between **evaluation** and **control**. We use both, for different jobs: the optimality equation to *find* a policy, the expectation equation to *score* every policy on a level field.

1.3 6.3 Why value iteration converges

1.3.1 6.3.1 The Bellman optimality operator and its contraction

Define $T : \mathbb{R}^{|\mathcal{S}|} \rightarrow \mathbb{R}^{|\mathcal{S}|}$ by

$$(TV)(s) = \max_{a \in \mathcal{A}(s)} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right].$$

Claim. T is a γ -contraction in the sup-norm $\|V\|_\infty = \max_s |V(s)|$:

$$\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty \quad \forall V, U \in \mathbb{R}^{|\mathcal{S}|}.$$

Proof. Fix s and let $a^\dagger = \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s')]$. Then

$$(TV)(s) - (TU)(s) \leq \gamma \sum_{s'} P(s' | s, a^\dagger) [V(s') - U(s')] \leq \gamma \sum_{s'} P(s' | s, a^\dagger) \|V - U\|_\infty = \gamma \|V - U\|_\infty,$$

where the first inequality is because $(TU)(s) \geq R(s, a^\dagger) + \gamma \sum_{s'} P(s' | s, a^\dagger) U(s')$ (the max is at least the value at $a^\dagger$), and the last equality uses $\sum_{s'} P(s' | s, a^\dagger) = 1$. The argument is symmetric in V, U , so it bounds $|(TV)(s) - (TU)(s)|$; taking the max over s gives the claim. The two ingredients are the *non-expansiveness of max* ($|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$) and the fact that P is a **probability** distribution — the row sums to one, which is why the discount γ survives undiluted. ■

1.3.2 6.3.2 Banach

$\mathbb{R}^{|\mathcal{S}|}$ with $\|\cdot\|_\infty$ is a complete metric space (it is finite-dimensional). The **Banach fixed-point theorem** (S. Banach, *Fundamenta Mathematicae* 3:133–181, 1922) then gives, in one shot:

1. T has a **unique** fixed point V^* with $TV^* = V^*$.
2. From **any** initialisation V_0 , the iterates $V_{k+1} = TV_k$ converge to V^* .
3. The convergence is geometric: $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$.

That fixed point is the optimal value function, and its greedy policy $\pi^*(s) = \arg \max_{a \in \mathcal{A}(s)} [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')]$ is optimal among **all** policies, including history-dependent and randomised ones (Puterman 1994, Thm 6.2.10). That last clause is worth pausing on: we searched only over the $\prod_s |\mathcal{A}(s)|$ stationary deterministic policies, and we get optimality over a vastly larger class for free. It is a consequence of the Markov property, not of the algorithm.

Three things follow that are useful and that people forget:

- **Initialisation does not matter for correctness.** We start at $V_0 = 0$. Any starting point converges. It matters only for speed.
- **Convergence is unconditional in P .** VI converges just as happily on a *wrong* P — it will confidently return the exact optimum of the wrong MDP. §6.9 is about what that costs.
- **$\gamma \rightarrow 1$ is the hard regime.** The rate is γ and the error constants below all carry $1/(1 - \gamma)$. At $\gamma = 0.99$ that factor is 100.

1.3.3 6.3.3 The stopping bounds — how we *choose* a tolerance instead of guessing

VI cannot see $\|V_k - V^*\|$ (it does not know V^*). It can see the **Bellman residual** $\varepsilon_k = \|V_{k+1} - V_k\|_\infty$. The two are linked. Write $V^* - V_{k+1} = \sum_{j \geq 1} (V_{k+j+1} - V_{k+j})$ and apply the contraction to each term:

$$\boxed{\|V_{k+1} - V^*\|_\infty \leq \frac{\gamma \varepsilon}{1 - \gamma}} \quad \text{whenever} \quad \|V_{k+1} - V_k\|_\infty < \varepsilon.$$

And the quantity we actually care about is not the value error but the **loss of the greedy policy** extracted from an inexact V . If $\|V - V^*\|_\infty \leq \delta$ and π_V is greedy w.r.t. V , then (Williams & Baird 1993; S. P. Singh & R. C. Yee, "An Upper Bound on the Loss from Approximate Optimal-Value Functions," *Machine Learning* 16:227–233, 1994, DOI [10.1007/BF00993308](https://doi.org/10.1007/BF00993308)):

$$\boxed{\|V^{\pi_V} - V^*\|_\infty \leq \frac{2\gamma \delta}{1 - \gamma}}$$

Chaining the two — substituting $\delta = \gamma \varepsilon / (1 - \gamma)$ — gives the residual-to-policy-loss bound $\|V^{\pi_k} - V^*\|_\infty \leq 2\gamma^2 \varepsilon / (1 - \gamma)^2$. (Both boxed forms appear in the literature and they are *not* the same statement: the first has the residual on the right, the second has the value error. Confusing them costs you a factor of $1/(1 - \gamma)$, which at $\gamma = 0.99$ is a factor of 100. We keep them separate.)

This is how the tolerance gets chosen, and it is the whole reason to state the bounds. We want the returned policy to be optimal to well inside the precision anyone could care about. We run `value_iteration` at $\text{tol} = 10^{-9}$. Then:

$$\|V_k - V^*\|_\infty \leq \frac{0.99 \times 10^{-9}}{0.01} = 9.9 \times 10^{-8}, \quad \|V^{\pi_k} - V^*\|_\infty \leq \frac{2 \times 0.99^2 \times 10^{-9}}{10^{-4}} \approx 2.0 \times 10^{-5}.$$

Against $|V^*| = 163.637$ that is a relative policy loss below 1.2×10^{-7} percent. So when we call VI's policy "the exact optimum" and use it as the denominator of every regret number, that is a claim with a certificate behind it, not a hope. Had we picked $\text{tol} = 10^{-3}$ "because it looked converged", the greedy-loss bound would only guarantee 2×10^1 — worthless, on a V^* of -163.6 . The lesson is that at γ near 1, eyeballing the residual curve is not enough; the $1/(1 - \gamma)^2$ amplification is real.

1.3.4 6.3.4 What we measured

VI converged in **2273 sweeps in 0.23 s**. The measured contraction rate — the geometric decay rate fitted to the residual sequence ε_k — is **0.9900**, i.e. γ to four decimal places.

That is not a coincidence and it is not a tautology either; it is the theory being tight. The contraction bound says the rate is *at most* γ ; whether it is *achieved* depends on the MDP. The rate is asymptotically governed by the sub-dominant structure of P_{π^*} , and for a chain that mixes slowly relative to the horizon — ours has a hard 24-hour periodicity and a slow evening outage process — the worst case is essentially attained. A rate strictly below γ would mean the chain forgets faster than the discount does; ours does not. So the observed 0.9900 is a small piece of empirical confirmation that the operator we implemented is the operator we derived, and we report it as such.

The sweep count is also predictable: from $V_0 = 0$ the first residual is $\varepsilon_0 = \max_s \max_a |R(s, a)| \approx 50$ (an empty tank at the evening demand peak with no grid and no fuel), so a rate- γ decay to 10^{-9} needs about $\ln(10^{-9}/50)/\ln(0.99) \approx 2.4 \times 10^3$ sweeps. We observed 2273. The gap is because the early sweeps contract slightly faster than the worst case, which is exactly what one would expect.

1.3.5 6.3.5 Policy iteration, for contrast

Source. Howard 1960 (as above).

Policy iteration alternates: (i) evaluate π exactly by solving $(I - \gamma P_\pi)V^\pi = R_\pi$ — the linear system of §6.2.1 — and (ii) improve, $\pi'(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^\pi]$. The **policy improvement theorem** says $V^{\pi'} \geq V^\pi$ componentwise, with equality iff π is already optimal. Since there are finitely many deterministic policies and each iteration strictly improves at least one state's value or halts, PI **terminates in finitely many iterations** with the exact optimum — typically a handful, often under ten. VI, by contrast, only converges asymptotically.

We did not use PI as the headline solver, and the honest reason is that VI at 0.23 s was already fast enough that the engineering effort had no payoff. Every ingredient PI needs is in the code anyway (policy_evaluation is PI's evaluation step). The trade is the usual one: PI does few, expensive iterations ($O(|\mathcal{S}|^3)$ dense, much less sparse); VI does many, cheap ones ($O(|\mathcal{S}| \cdot |\mathcal{A}| \cdot \text{nnz-per-row})$ per sweep). At γ close to 1, VI's sweep count blows up as $1/(1 - \gamma)$ and PI's iteration count barely moves — so at $\gamma = 0.999$ the ranking would likely flip.

1.4 6.4 Model-based vs model-free, stated sharply

The distinction is about **what the algorithm is allowed to read**, and nothing else.

- **Model-based** requires $P(s' | s, a)$ and $R(s, a)$ **written down** — an object you can sum over. Value iteration's backup contains the literal term $\sum_{s'} P(s' | s, a) V(s')$. That sum is

not computable unless somebody hands you the distribution. In our code that “somebody” is `build_model`, and VI’s entire interface to the world is the pair (P, R) .

- **Model-free** requires only **samples**: put in (s, a) , get back one (s', r) . Our `step()` is a four-line function that returns a sampled successor and a realised reward and nothing else. Q-learning calls it and never sees a probability.

That is the whole definition. It is a statement about **information access**, not about algorithm quality, and the sloppy version of this distinction (“model-free is more modern / more general / better”) is the single most common error in undergraduate RL writing.

Three consequences that follow immediately and that our experiments demonstrate:

1. **With a correct model, planning wins by construction.** VI returns the exact optimum. No amount of sampling can beat exact. Asking “does Q-learning beat VI?” with a correct P is asking whether an approximation beats the thing it approximates. It does not, and if it appeared to, we would have a bug.
2. The interesting question is therefore not *which wins* but **how wrong the model has to be, and how much experience you have to buy, before learning beats planning.** Dhaka publishes a load-shedding schedule that is famously optimistic. This is not a hypothetical framing device.
3. **Sample access is strictly weaker than model access, but strictly stronger than nothing** — and crucially, samples can be *converted into* a model (§6.9). The moment you notice that, the model-free/model-based line stops being a fence and starts being a dial.

We enforce the fence in code rather than in prose. `HallWaterMDP` has two faces, `build_model()` and `step()`, and the parity test Monte-Carlo-estimates $\hat{P}$ and $\hat{R}$ back out of `step()` and asserts they agree with `build_model()` within sampling error. If that test fails, every VI-vs-QL number in the report is meaningless, so it is the gate the whole experiment hangs on.

1.5 6.5 Q-learning

Source. C. J. C. H. Watkins, *Learning from Delayed Rewards*, PhD thesis, King's College, Cambridge, 1989; C. J. C. H. Watkins & P. Dayan, “Q-learning,” *Machine Learning* 8(3-4):279–292, 1992, DOI [10.1007/BF00992698](https://doi.org/10.1007/BF00992698). The TD idea it rests on is R. S. Sutton, “Learning to Predict by the Methods of Temporal Differences,” *Machine Learning* 3:9–44, 1988, DOI [10.1007/BF00115009](https://doi.org/10.1007/BF00115009).

1.5.1 6.5.1 The update

$$Q(s, a) \leftarrow Q(s, a) + \alpha_n(s, a) \left[\underbrace{r + \gamma \max_{a' \in \mathcal{A}(s')} Q(s', a')}_{\text{TD target}} - Q(s, a) \right],$$

where $\alpha_n(s, a) \in (0, 1]$ is the step size on the n -th visit to (s, a) , and the bracketed quantity is the **TD error** δ_t . Read the update as stochastic approximation: the TD target is a **one-sample unbiased estimate** of the Bellman optimality backup,

$$\mathbb{E}_{s' \sim P(\cdot | s, a), r} \left[r + \gamma \max_{a'} Q(s', a') \right] = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q(s', a') = (TQ)(s, a).$$

So Q-learning is Robbins–Monro applied to the fixed-point equation $Q = TQ$ — the same operator whose contraction we proved in §6.3, approached with noisy samples instead of exact expectations. That is the entire idea, and it is why the convergence conditions in §6.6 look like they do: they are the conditions under which the noise averages out fast enough to let the contraction do its work.

1.5.2 6.5.2 Why it is off-policy

The agent **behaves** with ε -greedy: with probability ε it picks uniformly from $\mathcal{A}(s)$, otherwise $\arg \max_a Q(s, a)$. Call that the **behaviour policy** μ . But the target contains $\max_{a'} Q(s', a')$ — the value of the **greedy** action at s' , which is the action the *target policy* $\pi = \text{greedy}(Q)$ would take, and which μ may well not take on the next step.

The learner bootstraps off an action it did not (necessarily) execute. That mismatch — target policy $\neq$ behaviour policy — is the definition of **off-policy**, and it is exactly what lets Q-learning converge to Q^* (the value of the optimal policy) while behaving sub-optimally for the entire duration of training. The exploration is charged to the behaviour policy; the learning is credited to the target policy; the two are decoupled. You could not do this on-policy: you would learn the value of the exploratory policy, which is not what you want.

1.5.3 6.5.3 SARSA, in two sentences

Source. G. A. Rummery & M. Niranjan, "On-line Q-learning using Connectionist Systems," Tech. Rep. CUED/F-INFENG/TR 166, Cambridge Univ., 1994; convergence in S. Singh, T. Jaakkola, M. Littman, C. Szepesvári, "Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms," *Machine Learning* 38:287–308, 2000, DOI [10.1023/A:1007678930559](https://doi.org/10.1023/A:1007678930559).

SARSA replaces the max with the action actually taken: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$ with $a' \sim \mu(\cdot | s')$. It is therefore **on-policy** — it learns Q^μ , the value of the ε -greedy policy it is actually running, and converges to Q^* only if μ is annealed to greedy (GLIE). The practical difference: SARSA accounts for the cost of its own exploration, so on problems where exploring is dangerous (the cliff-walk being the canonical example) it learns a *safer* policy than Q-learning, which happily learns the value of an optimal path it never dares walk. In our hall, exploration costs money but nothing irreversible happens, so the off-policy choice is free and we take it.

1.6 6.6 Convergence conditions for Q-learning

1.6.1 6.6.1 The two conditions

Tabular Q-learning converges to Q^* with probability 1 provided:

(C1) Robbins-Monro step sizes. For every (s, a) ,

$$\sum_{n=1}^{\infty} \alpha_n(s, a) = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \alpha_n^2(s, a) < \infty.$$

The first condition says the steps are large enough, in aggregate, to travel any finite distance — the estimate can still move no matter how far it has to go, so an unlucky initialisation is recoverable. The second says they shrink fast enough for the injected noise (variance $\propto \alpha_n^2$) to be summable — so the estimate eventually stops jittering. Both must hold. (H. Robbins & S. Monro, "A Stochastic Approximation Method," *Annals of Mathematical Statistics* 22(3):400–407, 1951, DOI [10.1214/aoms/1177729586](https://doi.org/10.1214/aoms/1177729586).)

(C2) Infinite visitation. Every legal (s, a) pair is visited infinitely often. Not "every state" — every *pair*. The max in the target is an argmax over $\mathcal{A}(s')$, and an action never tried has whatever value we initialised it to; the argmax over a set containing an unexamined element is not an estimate of anything.

Given (C1) and (C2), plus bounded rewards and $\gamma < 1$: $Q_t \rightarrow Q^*$ almost surely.

Sources.

- C. J. C. H. Watkins & P. Dayan, "Q-learning," *Machine Learning* 8:279–292, 1992, DOI [10.1007/BF00992698](https://doi.org/10.1007/BF00992698) — the original proof, via an "action-replay process" construction.
- T. Jaakkola, M. I. Jordan, S. P. Singh, "On the Convergence of Stochastic Iterative Dynamic Programming Algorithms," *Neural Computation* 6(6):1185–1201, 1994, DOI [10.1162/neco.1994.6.6.1185](https://doi.org/10.1162/neco.1994.6.6.1185).

[10.1162/neco.1994.6.6.1185](#) — the clean stochastic-approximation proof; Q-learning and TD(λ) both fall out of one lemma about contractive stochastic iterations.

- J. N. Tsitsiklis, "Asynchronous Stochastic Approximation and Q-learning," *Machine Learning* 16:185–202, 1994, DOI [10.1007/BF00993306](#) — handles the **asynchronous** case, which is the one that actually applies: real Q-learning updates one (s, a) per step, not all of them, and different pairs get wildly different numbers of updates. This is the citation that licenses what our code does.
- E. Even-Dar & Y. Mansour, "Learning Rates for Q-Learning," *Journal of Machine Learning Research* 5:1–25, 2003 — moves from "it converges" to **how fast**, as a function of the step-size schedule. They show a linear rate $\alpha_n = 1/n$ can be exponentially slow in $1/(1 - \gamma)$, while a **polynomial** rate $\alpha_n = 1/n^\omega$ with $\omega \in (1/2, 1)$ gives polynomial-time convergence. This paper is the reason our default is $\omega = 0.7$ and not $1/n$.

1.6.2 6.6.2 Why a constant α cannot converge to Q^*

Take $\alpha_n \equiv \alpha$. Then $\sum \alpha_n = \infty$ (C1a holds) but $\sum \alpha_n^2 = \sum \alpha^2 = \infty$ (C1b fails). The failure is not a technicality; it is visible in one line of algebra. Unrolling the update,

$$Q_{t+1}(s, a) = (1 - \alpha) Q_t(s, a) + \alpha [r_t + \gamma \max_{a'} Q_t(s', a')],$$

the estimate is an **exponentially-weighted moving average** of its recent targets with effective window $\approx 1/\alpha$. It never stops averaging over a *fixed-size* window, so the variance of the noise in the targets never gets averaged away — it is asymptotically driven down only to a floor proportional to α . Formally, Q_t converges in distribution to a random variable concentrated in a **neighbourhood** of Q^* whose radius is $O(\sqrt{\alpha} \sigma / (1 - \gamma))$ for target-noise scale σ , and it keeps rattling around inside that neighbourhood forever. It does *not* converge to Q^* . It cannot: an estimator that permanently gives weight α to a single fresh noisy sample cannot have vanishing variance.

The trade is real, not a defect: a constant α never forgets how to move, which is precisely what you want in a **non-stationary** environment. Our hall is stationary, so we pay the cost of a constant α for none of its benefit.

We do not merely assert this. The hyperparameter sweep shows it:

step size	final regret vs V^*
polynomial, $\alpha_n = 1/(1 + n)^{0.7}$	25.1%
constant, $\alpha = 0.1$	50.4%

The constant- α run flattens out short of the optimum and stays there; the polynomial one keeps closing. (These sweep runs use a smaller training budget than the headline run in §6.9, so the absolute regrets are higher than the 15.13% quoted there. Only the ordering is the claim.) Our implementation counts visits per (s, a) and sets $\alpha_n = 1/(1 + n(s, a))^{0.7}$, which satisfies (C1) with $\omega = 0.7 \in (0.5, 1]$ and lands in Even-Dar & Mansour's polynomial regime.

1.7 6.7 Exploration vs exploitation in RL

This is the same axis as Part 1 §1.4, transposed. In a metaheuristic, exploration is *sampling far from the incumbent*; in RL it is *taking an action whose value you are not yet sure of*. The tension is identical: you cannot improve an estimate you never sample, and you cannot exploit an estimate you never trust.

The difference is that in RL, exploration has a **cost that is paid in the same currency as the objective** (every exploratory hour of diesel is real money) and, more subtly, exploration is what condition (C2) demands. Exploration in RL is not a heuristic nicety; it is a hypothesis of the convergence theorem.

1.7.1 6.7.1 ε -greedy

With probability ε take a uniformly random legal action; otherwise take $\arg \max_a Q(s, a)$. We anneal ε linearly from 1.00 to 0.05 over the first 60% of training and hold it there. Annealing to *zero* would be wrong for two reasons: it violates (C2) exactly (an action stops being visited), and a small floor is cheap insurance against a Q estimate that got unlucky early. Keeping $\varepsilon \geq 0.05$ is what GLIE conditions formalise for on-policy methods; off-policy Q-learning needs only (C2), which the floor delivers.

ε -greedy is not clever. It explores *uniformly* — as willing to re-try an action it has tried ten thousand times as one it has tried twice. Count-based and optimism-under-uncertainty methods (UCB-style bonuses, R-MAX) explore in proportion to *uncertainty* and have far better sample-complexity bounds. We use ε -greedy because it is what the syllabus specifies and because our state space is small enough that the difference does not decide the experiment. On a larger MDP it would.

1.7.2 6.7.2 Optimistic initialisation — and why $Q_0 = 0$ is optimistic *here*

Initialise $Q_0(s, a)$ above any achievable value. Then every untried action looks better than it is, the greedy step *itself* is exploratory, and the agent systematically works through untried actions until their estimates fall to reality. Optimism converts exploration from a random perturbation into a directed one, for free.

In our MDP every reward is ≤ 0 (they are costs: pump cost, overflow, shortage). So the true $Q^*(s, a) \leq 0$ everywhere, and initialising $Q_0 = 0$ is **optimistic by construction** — an untried action is quietly credited with the best value physically possible. That is doing real exploration work for us, and pretending otherwise would be taking credit that belongs to the sign of the reward function rather than to the algorithm. So we say so, and we expose `q_init` as a config knob so it can be ablated rather than silently benefiting from it.

1.7.3 6.7.3 Exploring starts

Each episode begins in a state drawn uniformly from $\mathcal{S}$. This is the crudest and most effective route to (C2). Without it, the agent starts every day at (half-full tank, midnight, grid up, full ration) and follows a near-greedy policy; states like “*tank full, 8am, no diesel left*” are then almost unreachable, their Q values never move off their initialisation, and the max over them in the TD target is bootstrapping off a number that means nothing. That poison propagates backwards through every state that can reach them.

Exploring starts buy the coverage that the convergence proof *assumes* and does not provide. The sweep:

exploring starts	final regret
on	25.1%
off	76.7%

A factor of three. This is the largest single effect in the entire hyperparameter sweep, larger than the step-size effect, and it is not about learning at all — it is about whether the algorithm's stated precondition is satisfied. Our headline run reaches **100% coverage of the 3432 legal (s, a) pairs** with a strictly positive minimum visit count; without exploring starts it does not.

The honest caveat: exploring starts require the ability to *reset the simulator into an arbitrary state*, which a real hall would not give you. It is a legitimate use of a simulator and an illegitimate assumption about the world, and any deployment story would have to replace it with something like an ε -floor plus a long enough run, or with directed exploration.

1.8 6.8 The discount factor as an effective horizon

1.8.1 6.8.1 $1/(1 - \gamma)$

In $G_t = \sum_k \gamma^k r_{t+k}$ the weight on a reward k steps ahead is γ^k . The weights sum to $\sum_{k \geq 0} \gamma^k = 1/(1 - \gamma)$, and their mean lag is $\sum_k k \gamma^k (1 - \gamma) = \gamma/(1 - \gamma)$. Either way the number $1/(1 - \gamma)$ is the natural unit: it is the **effective horizon**, the number of steps the agent meaningfully sees. (A cleaner statement: a discounted infinite-horizon MDP is equivalent to an undiscounted one with a $(1 - \gamma)$ per-step termination probability, whose expected length is exactly $1/(1 - \gamma)$.)

Our step is **one hour**. So:

γ	$1/(1 - \gamma)$	effective lookahead
0	1	1 hour — this hour only
0.9	10	10 hours — under half a day
0.99	100	100 hours — about four days
0.999	1000	six weeks

We chose $\gamma = 0.99$, and the reason is structural rather than aesthetic: the entire story of this MDP is *“bank cheap grid water in the afternoon against an evening outage you cannot see yet, and do not burn the diesel ration before you need it”*. That reasoning chain spans the 24-hour cycle. An agent with a lookahead shorter than one day literally cannot represent the trade-off, because the consequence it is trading against falls outside its horizon. At $\gamma = 0.99$ the lookahead is 100 hours, comfortably past the 24-hour cycle with margin to spare, and the value of a decision made at 2pm still carries measurable weight at midnight ($0.99^{10} = 0.90$).

The myopic control proves this is not decoration. `policy_myopic` is value iteration at $\gamma = 0$: it maximises this hour's expected reward and ignores the future entirely. That is not a strawman — it is the formally correct greedy policy, the exact optimum of a one-hour horizon. It scores **120.62% regret**: more than twice as costly as optimal. If it had landed near V^* , the problem would have had no long-horizon structure and the whole assignment would have been vacuous. Reporting that gap is how we prove it is not.

For contrast, the tuned two-threshold caretaker rule (pump on grid below θ_g , burn diesel below θ_f , both thresholds grid-searched to give the heuristic its best possible shot) reaches **14.51% regret**. It has the same action set as π^* , including the generator; what it cannot do is **read a clock**. It does not know that 6pm is coming. That single missing feature — temporal anticipation, which is precisely what a non-zero γ buys — is worth 14.5% of the objective.

1.8.2 6.8.2 Discounted vs average-reward, honestly

Our task is **continuing**: the hour wraps, there is no terminal state, the hall runs forever. For a continuing task there are two defensible optimality criteria, and they are not the same one:

- **Discounted**: maximise $\mathbb{E}[\sum_k \gamma^k r_k]$. Well-posed for any bounded r ; the contraction argument works; the optimal stationary policy exists and is computable. This is what we do.
- **Average-reward (gain-optimal)**: maximise $g^\pi = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}[\sum_{k < T} r_k]$. This is arguably the *more natural* objective for a hall that will still be a hall next year. Nobody running a residential building genuinely believes an outage in 2027 matters 0.99^{17520} times less than one today. The discount is a mathematical convenience that we are importing into a problem that did not ask for it.

The honest position, which we state up front in the code and repeat here: **the discounted-optimal policy is not guaranteed to be gain-optimal**. It is guaranteed to be gain-optimal only in the *Blackwell* limit — there exists $\gamma^* < 1$ such that for all $\gamma \in (\gamma^*, 1)$ the discounted-optimal policy is also gain-optimal (D. Blackwell, “Discrete Dynamic Programming,” *Annals of Mathematical Statistics* 33(2):719–726, 1962) — but γ^* is not known for our MDP and we did not compute it. Average-reward methods (R-learning; see S. Mahadevan, “Average Reward Reinforcement Learning: Foundations, Algorithms, and Empirical Results,” *Machine Learning* 22:159–195, 1996, DOI [10.1007/BF00114727](https://doi.org/10.1007/BF00114727)) optimise g directly and would be the principled choice.

We chose discounted for three defensible reasons and one weak one. The defensible: (i) it is what the assignment specifies and what the contraction theory in §6.3 requires; (ii) $1/(1-\gamma) = 100$ h genuinely does exceed the horizon on which the problem's structure lives, so we are not truncating anything that matters; (iii) it gives a unique optimal stationary policy without any of the unichain/multichain assumptions that average-reward DP needs. The weak one: it is easier. We sweep γ in the experiments and report the resulting policies rather than asserting insensitivity.

One consequence we are careful about in the plots: we never draw the discounted V^* line on an average-cost axis. `rollout_stats` reports taka-per-day and dry-hours-per-day because that is what a hall warden asks about, but those are *translations* of the policy into consequences, not the objective the policy was optimised for. They are different optima for different criteria, and overlaying them would be a category error.

1.9 6.9 Certainty equivalence: the bridge, and the headline result

1.9.1 6.9.1 The method

Estimate the model from data by maximum likelihood, then plan on the estimate:

$$\hat{P}(s' | s, a) = \frac{N(s, a, s')}{N(s, a)}, \quad \hat{R}(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s, a)} r_i,$$

then run value iteration on $(\hat{P}, \hat{R})$ and return its greedy policy. This is **certainty equivalence** — so named because you plan as if your point estimate were certain. (See L. P. Kaelbling, M. L. Littman, A. W. Moore, "Reinforcement Learning: A Survey," *JAIR* 4:237–285, 1996, DOI [10.1613/jair.301](https://doi.org/10.1613/jair.301), §"model-based methods"; the ideas trace to adaptive control.)

Our `certainty_equivalence` builds $\hat{P}$ and $\hat{R}$ from **the counts Q-learning itself accumulated**, on the same training run, from the same samples, in the same order. Nothing extra is collected. Unvisited (s, a) pairs — of which there are none at full budget, but there are early in training — get a **pessimistic** treatment: a self-loop at the worst reward the MDP can physically produce, $-(c_{\text{gen}} + c_{\text{ovf}} \cdot \text{pump} + c_{\text{short}} \cdot D_{\text{max}}) = -127.5$. That is deliberately the opposite choice from Q-learning's optimistic $Q_0 = 0$: an optimistic *planner* would fall in love with an action it never tried and confidently recommend it, which is the classic way certainty equivalence goes wrong.

1.9.2 6.9.2 The result

On the **same 720,000 samples**:

method	regret vs V^* (mean $\pm$ s.d. over seeds)
Q-learning	15.13% $\pm$ 2.36
Certainty-equivalence VI	1.43% $\pm$ 0.23

A **factor of ten**, on identical data. This is the headline finding of the decision-making half, and the explanation is not subtle once you look at where a sample *goes*.

1.9.3 6.9.3 Why the learned model is so much more sample-efficient

Think about what each algorithm does with one transition (s, a, r, s') .

Q-learning performs one update, to one cell: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. Then it throws the sample away. The information in that transition reaches the rest of the table only *later*, and only by diffusion: it must be carried backward one bootstrap hop per visit, through states that themselves have to be re-visited to move. Value propagates through the table at roughly one step of horizon per pass over the data. With an effective horizon of 100 and a γ of 0.99, information has to make a great many hops before it is felt where it matters — and

every hop is re-scaled by a learning rate that is itself decaying. This is why the standard sample-complexity bound for tabular Q-learning carries $1/(1 - \gamma)^4$: one factor from the horizon in the value scale, one from the variance, and two more from the sequential propagation.

Certainty equivalence takes the same transition and increments a counter, $N(s, a, s') += 1$. That counter is then **read by every single Bellman backup, of every sweep, forever**. VI on $\hat{P}$ performed 2273 sweeps; each sweep touches every (s, a) row of $\hat{P}$; so a single observed transition is *re-used thousands of times*, and its implications are propagated to every state that can reach (s, a) — not by diffusion through the data, but by the planner, which is allowed to do arbitrarily much computation on a fixed dataset.

That is the entire asymmetry, and it is a **compute-for-samples trade**. A Q-learning update spends a sample on one cell and discards it. A learned model *stores* the sample as a sufficient statistic and lets planning amortise it across every backup. Samples are expensive (they are hours of a real hall); backups are free (0.23 s). We are trading the cheap resource for the expensive one, and the factor of ten is the exchange rate.

The theory agrees. The minimax sample complexity of learning an ε -optimal policy with a generative model is $\tilde{O}(|\mathcal{S}||\mathcal{A}|/((1 - \gamma)^3 \varepsilon^2))$, and it is achieved by the **model-based** estimator (M. G. Azar, R. Munos, H. J. Kappen, "Minimax PAC Bounds on the Sample Complexity of Reinforcement Learning with a Generative Model," *Machine Learning* 91:325–349, 2013, DOI [10.1007/s10994-013-5368-1](https://doi.org/10.1007/s10994-013-5368-1)). The corresponding bounds for tabular Q-learning are worse by a factor of $1/(1 - \gamma)$ (Even-Dar & Mansour 2003; and see C. Jin, Z. Allen-Zhu, S. Bubeck, M. I. Jordan, "Is Q-Learning Provably Efficient?", *NeurIPS* 2018, for the regret-minimising analogue). At $\gamma = 0.99$ that missing factor is 100. We did not tune our way to a factor of ten; the factor of ten was there in the bounds all along.

1.9.4 6.9.4 What this forces us to conclude — and what it forbids

This baseline is in the experiment precisely because **it could have sunk the thesis**, and it partly did.

Had we run only VI-with-the-true-model against Q-learning, we could have told a clean story: *"the model is wrong in the real world, so model-free wins."* The certainty-equivalence arm makes that story unavailable. If a model **learned from n samples** beats Q-learning **trained on the same n samples** by a factor of ten, then "model-free beats model-based" is simply the wrong lesson. The right one is narrower and truer:

A **wrong prior model** loses to a **learned model**. It does not lose to model-free learning. You reach for model-free methods when you cannot even write down the state-transition *structure* to estimate — not merely when your numbers are wrong.

That is a weaker claim than the one we would have liked to make, and it is the one the data supports. Writing the stronger one would have been the easier report and the dishonest one.

1.9.5 6.9.5 How wrong does a prior model have to be?

The mis-specification sweep gives the planner a wrong outage_scale (a multiplier on p_{fail} only — "outages happen twice as often" and "outages last twice as long" are different errors with different consequences, so we vary exactly one) and then scores the resulting policy under the **true** model:

prior model	regret vs V^*
outages 1.25× rarer than reality	0.11%
outages 4× rarer than reality	2.66%
"the grid never fails" ($p_{\text{fail}} = 0$)	21.39%
Q-learning, 720k samples	15.13%
certainty-equivalence VI, same 720k samples	1.43%

Read that table carefully, because it is the answer to the question the assignment actually asks.

Planning with a model that is **wrong by a factor of four** still beats Q-learning trained on 720,000 samples (2.66% vs 15.13%). Value iteration is remarkably tolerant of a mis-specified P — a mild

surprise to us, and the reason is that the *ordering* of actions is what determines the policy, and moderate perturbations of P do not reorder the argmax in most states. The greedy operator is a step function; it does not care about small changes in its input.

The prior model only loses when it is **structurally** wrong. A planner told "the grid never fails" does not merely mis-price the outage risk; it **never learns that diesel exists as a hedge**, because in its world the hedge protects against an event of probability zero. That policy scores 21.39% — worse than Q-learning, and worse than the hand-tuned threshold rule. The failure mode is qualitative, not quantitative.

So: how wrong does your model have to be before learning beats planning? Roughly — **wrong enough to have deleted a phenomenon, not merely wrong enough to have mis-measured one**. And even then, the right response is not to abandon models; it is to *learn* one (1.43%).

1.10 6.10 Side-by-side summary table

	Value Iteration	Policy Iteration	Q-learning	SARSA	Certainty Equivalence
What it needs	explicit $P(s' s, a)$, $R(s, a)$	same as VI	samples (s, a, r, s') only	samples (s, a, r, s', a') only	samples, then builds $\hat{P}$, $\hat{R}$, then needs a planner
What it guarantees	$V \rightarrow V^*$ geometrically at rate γ ; unique fixed point (Banach); certified via $\ V_k - V^*\ \leq \gamma \varepsilon / (1 - \gamma)$	exact π^* in finitely many iterations (monotone improvement)	$Q \rightarrow Q^*$ a.s. under Robbins-Monro + infinite visitation; off-policy	$Q \rightarrow Q^\mu$; $\rightarrow Q^*$ only if μ is GLIE; on-policy	$\hat{P} \rightarrow P$ by LLN, so $\pi \rightarrow \pi^*$; no guarantee at finite n (optimism bias)
Sample complexity	zero samples; $O(\text{sweeps} \times \mathcal{S} \mathcal{A} \cdot \text{nnz})$ compute. Ours: 2273 sweeps, 0.23 s	zero samples; few iterations, each an $ \mathcal{S} $ -dim linear solve	worst among these; bounds carry $1/(1 - \gamma)^4$. Ours: 720k steps $\rightarrow$ 15.13%	same order as Q-learning, usually a little worse (learns the exploratory policy's value)	minimax-optimal, $\tilde{O}(\mathcal{S} \mathcal{A} / ((1 - \gamma)^3 \varepsilon^2))$. Ours: same 720k $\rightarrow$ 1.43%
When to use	you have a trustworthy model and $ \mathcal{S} $ fits in memory	same, and γ is close to 1 (VI's sweep count blows up; PI's does not)	you have a simulator or a live system and cannot write down the transition structure at all	same, but exploration is <i>dangerous</i> and you want the policy to price its own risk	you have samples and you can write down the state space — i.e. almost always, in tabular problems
Main limitation	needs P ; $O(\mathcal{S} \mathcal{A})$ memory; converges to the exact optimum of whatever model you gave it , wrong or not	$O(\mathcal{S} ^3)$ per evaluation if dense; still needs P	sample-hungry; every sample updates one cell then is discarded; needs (C2) coverage, which needs exploring starts or something like them	slower to Q^* ; conservative by construction (a feature on cliffs, a bug elsewhere)	needs $O(\mathcal{S} ^2 \mathcal{A})$ counts (does not scale to large $\mathcal{S}$); overconfident on unvisited (s, a) unless you add pessimism, which we do

Cross-cutting reading. All five are the same fixed-point equation approached from different information positions. VI and PI apply the Bellman operator with **exact expectations** because they were handed P . Q-learning and SARSA apply it with **one-sample estimates** because they

were not. Certainty equivalence **reconstructs** enough of P from samples to go back to exact expectations, which is why it dominates in our data. The apparent dichotomy “model-based vs model-free” is really a dial marked *how much of the model do you have, and how much are you willing to estimate*.

1.11 6.11 Where this connects to the swarm half

The two halves of this lab look unrelated — a particle swarm placing Wi-Fi access points, a caretaker deciding when to run a pump — and they are the same problem seen from two angles.

Both are search under a budget. PSO searches a continuous space $\mathcal{S} \subseteq \mathbb{R}^6$ with a population of 30 candidate solutions and a budget counted in **objective-function evaluations**. Value iteration searches the policy space $\prod_s \mathcal{A}(s)$ — combinatorially enormous, 3^{1584} before the availability mask — with dynamic programming and a budget counted in **Bellman backups**. Q-learning searches the same policy space with a budget counted in **environment samples**, which is the most expensive currency of the three. In every case the algorithm is a strategy for spending a finite resource to reduce uncertainty about where the optimum is.

The exploration/exploitation axis is literally the same axis. Part 1 §1.4 defines it in terms of population diversity $D(P_t)$ collapsing over a run; §6.7 defines it in terms of ε annealing and optimistic Q_0 . Both are schedules that start broad and narrow. Both fail the same way: PSO stagnates when velocities collapse before x^* is found; Q-learning’s argmax freezes on a badly-estimated action when ε decays before every (s, a) has been sampled. Premature convergence is one phenomenon with two names.

They differ in what structure they exploit, and that difference is the whole reason both are on the syllabus. PSO assumes *fitness locality* — nearby points have correlated fitness (Part 1 §1.7.2) — and nothing else. It cannot use structure it does not have. Value iteration assumes the *Markov property* and, given it, extracts an exact answer with a certificate. When you have the Markov structure, throwing a swarm at the policy space would be absurd: you would be using a method designed for the absence of structure on a problem that has it in abundance. The converse holds too — our Wi-Fi objective has no Markov decomposition, no state, no transition kernel, and dynamic programming has nothing to bite on.

NFL applies to both, and in the same way. Wolpert & Macready (Part 1 §1.3) say that averaged over all objective functions, no optimiser beats another. The RL statement of the same fact: averaged over all MDPs, no learning algorithm beats another. Q-learning’s dominance over random search is not a free lunch — it is bought with the assumption that the environment is Markov and stationary, exactly as PSO’s dominance over random search is bought with the assumption that fitness is locally correlated. **Each algorithm is a bet on a structural prior, and it wins exactly to the extent that the bet is right.**

Our results are that principle made numerical. VI wins by construction when the Markov model is correct — the bet is exactly right. It degrades gracefully to 2.66% when the model is wrong by $4\times$ — the bet is approximately right. It collapses to 21.39% when told “the grid never fails” — the bet is *structurally* wrong, a phenomenon has been deleted from the prior, and no amount of exact computation on a wrong model recovers it. Q-learning, which bets on almost nothing beyond Markovianity, is never catastrophic and never excellent: 15.13% at 720,000 samples. And certainty equivalence, which bets on the Markov structure but declines to bet on the *numbers*, gets 1.43% — the best of both, because it made the one assumption that is true and estimated the rest.

That is the same lesson Part 1 ends on, arrived at from the other side: **match the prior to the problem, and be honest about which prior you are actually assuming.**